guildtrek

PYTHON FOR AI AND DATA ENGINEERING · DAY 3

Day 3, Maintainable

Code is read and changed far more than it is written. We know every construct; we use one only when it fits.

Paradigms and their constructs, the same constructs in Python, then a simple rule for when to use each one, tested on one running example.

What we will learn today

THE SEVEN QUALITIES — Performant · Scalable · Reliable · **Maintainable** · Interoperable · Trustworthy · Secure

TODAY maintainable. Easy to read, easy to change, for years.

Part 1 PARADIGMS AND CONSTRUCTS	Dynamic, object-oriented, functional, and metaprogramming: the tools each style gives you.
Part 2 THE SAME, IN PYTHON	How Python writes each construct, in the same order, on a real reviews service.
Part 3 USE IT ONLY IF IT FITS	When to use each construct and when not to, with SOLID, patterns, and a rough-to-clean walk.

HOW EACH IDEA LANDS — A teaching slide for the concept, then a demo that shows its code and proves it, then we build it for real on reviews-insight.

The one rule

A language gives you many **constructs**: a class, a function, a decorator. We know them all. The rule that keeps code maintainable is simple.

Do not use a construct just because you know it.

Each construct helps in some cases and only adds weight in others. Before you reach for one, ask a single question:

> THE QUESTION Does this make the code easier to read and change? One example runs through the whole day, a **reviews service**, and we ask it of every construct: better, or worse?

PART 1 · PARADIGMS AND THEIR CONSTRUCTS

Four styles, and the tools they give you

A paradigm is a way of organizing code. No paradigm is the one right answer; most real code mixes several, and Python supports them all.

Construct, paradigm, and the baseline

Construct

A building tool the language gives you: a variable, a function, a class, a decorator.

Paradigm

A style of putting constructs together to organize a whole program. A way of thinking about code.

The baseline everyone starts with is **procedural**: statements in order, grouped into functions, top to bottom, like a recipe. The four paradigms below are extra tools for when plain procedural code gets hard to manage.

Dynamic | Object-oriented | Functional | Metaprogramming

Dynamic

Types are decided while the program runs, not before. Two constructs.

Dynamic binding

A name is a label you can put on any value, and move to a value of another type. Less setup, more general code.

Duck typing

"If it walks like a duck and quacks like a duck, it is a duck." Call the method you need; any object that has it works, whatever its class.

HELPS THEN HURTS Easy to extend and to test (pass a fake). The risk: a wrong object is caught only when the line runs. The fix: type hints plus a checker (mypy). SOLID link: this is the everyday form of Dependency Inversion.

Object-oriented

Bundle data with the behaviour that works on it, into an object. The key choice is how two objects connect.

Inheritance: ”is-a”

One class builds on another. A `SavingsAccount` **is a** `BankAccount`. Useful for a true, stable is-a, but deep chains turn rigid.

Composition: ”has-a”

An object holds another and uses it. A `Car` **has an** `Engine`, which you can swap. The flexible default.

Prefer composition over inheritance. If you are inheriting only to reuse a method, use composition instead. Polymorphism: many classes with the same method (`area()`) let one function work with all of them.

`SOLID LIVES HERE` Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion. In Python the first and last do most of the work.

Functional

Build the program out of small functions that take input and return output, and avoid changing shared data.

Pure functions

Same input, same output, and nothing else changes. The easiest code to read and test.

Immutability

Data that cannot change after it is made. Nothing alters it behind your back; safe to share.

Functions as values

Pass a function to another function (a higher-order function), so one general piece takes the varying part in.

THE BALANCE A real program must do side effects (files, printing). Keep the core pure and immutable, and push the input and output to the edges.

Metaprogramming: use it sparingly

Code that works on code: it inspects, changes, or generates other code. Powerful, and easy to overuse.

Decorator

Wrap a function to add behaviour around it (timing, logging) without changing it. The one you will use.

Property / descriptor

Run a method when you read what looks like a plain attribute. Handy in small doses.

Metaclass

A blueprint for classes. The rarest tool; you almost never need one.

THE RISK Each tool hides behaviour behind something that does not look like a normal call. Use it only for a real cross-cutting concern, and reach for the plainest tool that does the job.

PART 2 · THE SAME CONSTRUCTS IN PYTHON

How Python writes each one

Python is multi-paradigm: it gives you all four styles, in the same order. Here we start building the reviews service.

Dynamic in Python

Python is dynamically typed: a name can point at any object. Duck typing means code depends on a method, not a class. Add type hints so mistakes show early.

```
def average(values: list[float]) -> float:   # hints: mypy checks before it runs
    return sum(values) / len(values)

class ReviewSource(Protocol):                 # "anything with rows()" fits, no base class
    def rows(self) -> Iterator[dict[str, str]]: ...
```

FLEXIBLE AND SAFE Duck typing keeps the code open to new kinds of source; a Protocol plus mypy catches a wrong one before the program runs.

Object-oriented in Python

class

Worth it when it holds state several methods share. A one-method class is usually just a function.

dataclass

Generates `__init__`, `__repr__`, `__eq__` from the fields; `frozen=True` makes a safe value object.

composition

A `Report` holds a formatter it can swap, instead of inheriting from a base report.

protocol

A small interface; any class with the right method fits, with no base class (structural typing).

DUNDER METHODS The special `__name__` methods Python calls for you: `__init__` (constructor), `__repr__` (developer text), `__eq__` (what `==` means), plus `__hash__`, `__len__`, `__enter__`/`__exit__`, and `__post_init__`.

Demo · class versus function

DEMO `poetry run python demo_class_vs_function.py`

The claim: a one-method class adds nothing a function does not; real shared state is what needs a class.

```
class Discounter:                       # one method plus a constructor
    def __init__(self, rate): self.rate = rate
    def apply(self, price): return price * (1 - self.rate)

def discounted(price, rate):            # the same thing, simpler
    return price * (1 - rate)
```

```
class    : 90.0     function: 90.0     a class is the right choice here: 15
```

WHAT IT MEANS The class and the function give the identical answer, so here the class is just heavier. A `Tally` that keeps a running total across calls is where a class is right.

Demo · dataclass

DEMO `poetry run python demo_dataclass.py`

The claim: a dataclass matches a hand-written class with far less code, and a frozen one cannot be changed.

```python
class PointManual:                      # __init__, __repr__, __eq__ by hand
    def __init__(self, x, y): ...
    def __repr__(self): ...
    def __eq__(self, other): ...

@dataclass
class Point:                            # the same, generated from two fields
    x: int
    y: int
```

```
manual   : PointManual(x=1, y=2) True
dataclass: Point(x=1, y=2) True
frozen point cannot be changed after construction
```

WHAT IT MEANS Same readable repr and value equality, from far less code. Adding `frozen=True` then rejects any later change with a `FrozenInstanceError`.

Demo · composition

DEMO `poetry run python demo_composition.py`

The claim: swapping the part a Report holds changes its behaviour with no change to Report itself.

```
class Report:
    def __init__(self, formatter):      # HAS a formatter (composition)
        self._formatter = formatter
    def render(self, rows):
        return self._formatter.format(rows)

Report(PlainFormatter()).render(rows)
Report(BulletFormatter()).render(rows)
```

```
plain  : alpha, beta, gamma
bullets: - alpha / - beta / - gamma
```

WHAT IT MEANS The same Report code produced a comma line and a bullet list, chosen entirely by the formatter handed in. Behaviour was composed, not inherited.

Demo · protocol

DEMO `poetry run python demo_protocol.py`

The claim: two unrelated classes satisfy one protocol by shape alone, and one function accepts either.

```
class Formatter(Protocol):
    def format(self, value: float) -> str: ...

class Money:                              # no shared base class
    def format(self, value): return f"Rs {value:.2f}"
class Percent:
    def format(self, value): return f"{value:.0f}%"

def show(f: Formatter, v: float) -> str: return f.format(v)
```

```
money   : Rs 1234.50        percent: 87%
```

WHAT IT MEANS Neither class inherits from Formatter; both just match its shape, so `show()` accepts either. Duck typing the type checker can verify.

Functional in Python

Functions are values. Comprehensions and generators transform data top to bottom. A frozen dataclass or a tuple gives immutability.

```
ratings = [r.rating for r in reviews]       # a comprehension: build a list
total = sum(r.rating for r in reviews)      # a generator: nothing held in memory

def parse_review(row: dict[str, str]) -> Review:   # a pure function
    ...   # same row in, same Review out, nothing else touched
```

ON THE REVIEWS EXAMPLE Turning a raw row into a `Review` is a small pure function: easy to test with no file and no setup.

Metaprogramming in Python

The one you use is the decorator. A small `@timed` adds a cross-cutting concern in one place, without touching the method.

```
@timed("summary")
def summarize(self) -> dict[str, int]:
    ...
```

RARELY NEEDED Properties make a method look like an attribute; metaclasses build classes. Reach for these only when nothing simpler works.

Demo · decorator

DEMO `poetry run python demo_decorator.py`

The claim: a decorator adds a shared concern in one place, without editing the function it wraps.

```python
def traced(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"-> calling {func.__name__}{args}")
        result = func(*args, **kwargs)
        print(f"<- {func.__name__} returned {result}")
        return result
    return wrapper

@traced
def add(a, b): return a + b
```

```
-> calling add(2, 3)
<- add returned 5
result: 5
```

WHAT IT MEANS The tracing wraps the call and lives in one place; `add()` itself was never touched. One place to change, instead of copies in twenty functions.

PART 3 · USE A CONSTRUCT ONLY IF IT FITS

Three quick checks, then the example

Before you use a construct, check it against a simple when and when-not rule. Three tables, then the reviews example from rough to clean.

When to use each OO construct

Construct	Use it when	Do NOT use it when
class	data plus behaviour, shared state, or many instances	one method and a constructor: use a function
dataclass	a group of values travel together	a throwaway pair: a tuple is lighter
frozen dataclass	the value must not change after it is made	it genuinely has to change in place
inheritance	a true, stable "is-a", or defining an interface	only to reuse a method: use composition
composition	one object needs another to do a job	(this is the default; prefer it)
protocol	you want to swap or fake an implementation	there is only ever one implementation
decorator	one cross-cutting concern shared by many functions	it would hide important control flow

SOLID, as a quick check

Principle	The do	The don't
Single Responsibility	one class or function, one job	a function that reads, transforms, and prints
Open-Closed	add new behaviour by adding code	edit tested code for every new case
Liskov Substitution	a subclass works anywhere its parent does	a subclass that breaks the parent's promises
Interface Segregation	small, focused interfaces	one big interface nobody fully uses
Dependency Inversion	depend on a protocol and inject it	build a file reader or client inside a class

> IN PYTHON Single Responsibility and Dependency Inversion do most of the work; the other three mostly follow once you prefer composition and small protocols.

Design patterns, only when they fit

Pattern	Helps when	Skip it when
Factory	object creation is complex or in one place	a plain constructor call would do
Strategy	you swap an algorithm (composition plus a protocol)	there is only one algorithm
Dependency injection	you want to test or swap a dependency	(rarely a mistake; usually worth it)
Singleton	you truly need exactly one shared instance	it is a global by another name: avoid it

`YAGNI` "You are not going to need it." Build the simplest thing that solves today's problem. An abstraction with a single use is a candidate to inline back.

The reviews example, from rough to clean

Start with the roughest version that works, then improve it one construct at a time, checking each change against the tables.

0 · one big function, a dict → 1 · frozen dataclass → 2 · pure parse function → 3 · separate the jobs → 4 · protocol + inject → 5 · one decorator

What got better

A fixed, explicit shape, a testable pure function, separated jobs, a swappable source, timing in one place.

What would be worse

A one-method `ParserService` class; a deep `BaseSource` to `CsvFileSource` tree. Both add weight for no gain.

Demo · dict versus dataclass

DEMO `poetry run python demo_dict_vs_dataclass.py`

The claim: a dict has no fixed shape, so typos hide; a dataclass has one declared, checkable shape. (Step 1 of the walk.)

```
as_dict = {"product_id": "P1", "rating": 5}
as_dict["ratng"]        # KeyError, only at run time
as_dict["ratng"] = 4    # silently accepted: the data drifts

review = Review(product_id="P1", rating=5)   # one declared shape
```

```
dict typo on read, found only at runtime: 'ratng'
dict silently accepted a junk key: True
dataclass access is explicit: P1 5
```

WHAT IT MEANS A mistyped key is a run-time surprise on read and silent junk on write. The dataclass names its fields once, and mypy flags a wrong field before the program runs.

Demo · dependency injection

DEMO `poetry run python demo_dependency_injection.py`

The claim: a service that receives its source can be run with a real one or a fake, with no change to its logic. (Step 4 of the walk.)

```
class BadService:                       # builds its own source: stuck with it
    def __init__(self): self._source = RealSource()

class GoodService:                      # receives its source: swappable
    def __init__(self, source): self._source = source
```

```
bad service (fixed source) : 60
good service (real source) : 60
good service (fake source) : 3
```

WHAT IT MEANS The same GoodService logic gave 60 with a real source and 3 with a fake, chosen only by what was injected. That is Dependency Inversion, and it is what makes it testable.

LIVE LAB · THE BUILD

reviews-insight, made maintainable

The clean version, built for real: a source protocol, two implementations, an injected service, one decorator, and a test that proves the seam.

Aspect 1 · a protocol and two sources

Depend on a shape (Part 1 dynamic, Part 2 protocol). Two independent classes fit it, with no base class.

```
class ReviewSource(Protocol):
    def rows(self) -> Iterator[dict[str, str]]: ...

class CsvReviewSource:                  # reads a file
    def rows(self) -> Iterator[dict[str, str]]:
        with open_records(self._path) as stream:
            yield from stream

class InMemoryReviewSource:             # a list, for tests
    def rows(self) -> Iterator[dict[str, str]]:
        return iter(self._rows)
```

COMPOSITION, NOT A TREE Two small classes, each doing one job.

Aspect 2 · an injected service, plus one decorator

The service receives its source (Dependency Inversion) and drives it through the shared counting logic. One `@timed` decorator adds timing in one place.

```python
class SummaryService:
    def __init__(self, source: ReviewSource) -> None:
        self._source = source            # injected, not constructed here

    @timed("summary")
    def summarize(self) -> dict[str, int]:
        return summarize_rows(self._source.rows())
```

SEPARATION OF CONCERNS The service knows nothing about files; the CLI is a thin composition root that builds the source and injects it.

Aspect 3 · prove the seam, and run it

Summarize the same data from a file and from memory, and assert the results match. Swapping the source without touching the service is the proof.

```
from_file = SummaryService(CsvReviewSource(path)).summarize()
from_memory = SummaryService(InMemoryReviewSource(rows)).summarize()
assert from_file == from_memory == {"reviews": 3, "products": 2}
```

8	2	0	clean
tests pass	sources, one service	logic changes to swap	mypy strict

```
poetry run reviews summary  ->  reviews : 5   products: 3
```

THE DESIGN MADE IT Small, swappable pieces instead of one tangled function, and the same numbers as before.

What we covered

Paradigms and Python

- Dynamic, object-oriented, functional, metaprogramming
- The same constructs in Python, in order
- is-a versus has-a; prefer composition
- Dunder methods; pure functions; the decorator

Judgment and the build

- When to use each construct, and when not to
- SOLID as a quick check; design patterns; YAGNI
- reviews-insight from rough to clean
- Every concept taught, its code shown, then built

THE REVIEW LENS Does each class need to exist? Could inheritance be composition? Are dependencies injected? Does any abstraction have more than one real use? Are reading, logic, and printing kept apart?

guildtrek

DAY 3 · COMPLETE

Use the tool that fits, and no more.

Know every construct, and pick the one that makes the code easier to read and change. Each idea taught, its code shown in a demo, then built for real and proven by swapping a source with no change to the logic.

Next: Day 4, Reliable. Validate at the edges and handle failures so bad data cannot crash the app.